



Let's go ahead with understanding cascade-conflict-check.yml file.

Explain line by line what each line does....

Here's `cascade-conflict-check.yml` line by line, focusing on the YAML and what the JS is doing conceptually.^[1]

```
name: Cascade Conflict Check
```

- Display name of the workflow in the Actions UI.

```
on:
  pull_request_target:
    types: [opened, reopened, synchronize]
```

- `on:` declares what events trigger this workflow.
- `pull_request_target:` runs in the context of the base repo (not the PR's fork), which allows safe use of repo secrets like `CASCADE_GITHUB_TOKEN` even when PRs come from forks.^[2]
- `types: [opened, reopened, synchronize]:`
 - `opened:` when a PR is first created.
 - `reopened:` when a previously closed PR is reopened.
 - `synchronize:` when new commits are pushed to the PR branch (mergeability may change).

```
concurrency:
  group: cascade-conflict-${{ github.event.pull_request.number || github.run_id }}
  cancel-in-progress: true
```

- `concurrency:` ensures only one run for a given key is active at a time.^[2]
- `group:` builds a concurrency group string:
 - If `github.event.pull_request.number` exists, group becomes like `cascade-conflict-123` (one group per PR).
 - If for some reason `pull_request` is missing, it falls back to `github.run_id` to still have a unique group.

- `cancel-in-progress: true`: if a new run starts for the same group, any previous in-progress run for that PR is automatically cancelled, so you don't have overlapping conflict checks for the same PR.

```
permissions:
  contents: read
  pull-requests: write
  issues: write
```

- Explicit token permissions for this workflow:
 - `contents: read`: read branches/commits if needed.
 - `pull-requests: write`: update PR fields if necessary.
 - `issues: write`: add labels and comments (PRs are issues under the hood).

```
jobs:
  check-cascade-pr:
    if: startsWith(github.event.pull_request.title, '[Cascade')
    runs-on: ubuntu-latest
```

- `jobs:` starts the jobs section.
- `check-cascade-pr`: the job ID.
- `if: startsWith(github.event.pull_request.title, '[Cascade')`
 - Job guard: only runs if the PR title begins with `[Cascade` (so `[Cascade] foo`, `[Cascade Final] bar`, etc.).
 - This prevents the workflow from touching non-cascade PRs.^[1]
- `runs-on: ubuntu-latest`
 - Uses a GitHub-hosted Ubuntu runner.

```
steps:
  - name: Re-check merge conflicts
    uses: actions/github-script@v7
    with:
      github-token: ${ secrets.CASCADE_GITHUB_TOKEN }
      script: |
        ...
```

- `steps:` ordered list of steps in this job.
- One step:
 - `name`: label in the Actions UI.
 - `uses: actions/github-script@v7`: runs JavaScript with Octokit (`github`), `core`, and context injected.^[2]
 - `with.github-token`: uses your PAT-style secret `CASCADE_GITHUB_TOKEN` for GitHub API calls.

- `script:` |:multi-line JS body; everything under it (indented) is JS.

Now the JS body, conceptually:

```
const owner = context.repo.owner;
const repo = context.repo.repo;
const prNumber = context.payload.pull_request.number;
```

- Pulls `owner`, `repo`, and `prNumber` from the event payload (the PR that triggered this run).^[1]

```
async function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}
```

- Small helper: `sleep(ms)` returns a Promise that resolves after `ms` milliseconds; used for polling.

```
let pr;
for (let i = 0; i < 6; i++) {
  const resp = await github.rest.pulls.get({
    owner,
    repo,
    pull_number: prNumber
  });
  pr = resp.data;
  if (pr.mergeable !== null) break;
  await sleep(2000);
}
```

- Polls the PR's details up to 6 times:
 - Calls GET `/repos/{owner}/{repo}/pulls/{prNumber}` each time.
 - Stores the result in `pr`.
 - Breaks early if `pr.mergeable` is no longer `null` (GitHub has finished computing mergeability).
 - Waits 2 seconds between attempts if still `null`.
- Net effect: wait up to about 10 seconds for GitHub to decide if the PR is mergeable.

```
core.info(`Cascade PR #${prNumber} mergeable = ${pr?.mergeable}, mergeable_state =`
```

- Logs mergeability info to the Actions log for debugging:
 - `mergeable` is a boolean or `null`.
 - `mergeable_state` is a string like `'clean'`, `'dirty'`, `'blocked'`, etc.

```
if (pr?.mergeable === false || pr?.mergeable_state === 'dirty') {
  try {
    await github.rest.issues.addLabels({
      owner,
```

```

    repo,
    issue_number: prNumber,
    labels: ['cascade-conflict']
  });
} catch (e) {
  core.warning(`Could not add conflict label. ${e.message}`);
}

await github.rest.issues.createComment({
  owner,
  repo,
  issue_number: prNumber,
  body: [
    `This cascade PR still has merge conflicts.`,
    `Downstream cascade PR creation remains paused until this PR is resolved and m
  ].join('\n')
});

return;
}

```

- Condition: if the PR is known to be non-mergeable:
 - `mergeable == false` OR `mergeable_state == 'dirty'` (typical “has conflicts” state).
- Then:
 - Tries to add the `cascade-conflict` label to the PR.
 - If label add fails, logs a warning but continues.
 - Adds a comment explaining:
 - The PR still has conflicts.
 - Cascade downstream PR creation is paused until this PR is resolved and merged.
 - `return`: stop the script early; nothing else to do.

```
core.info(`Cascade PR #${prNumber} is not currently marked as conflicted.`);
```

- If the `if` block above did not execute:
 - Logs that the PR is not currently in a conflicting state (either clean, blocked, pending, etc., but not `dirty` / `mergeable == false`).
 - The workflow then ends without changing labels or comments.

So in plain language:

- This workflow runs on PR events for cascade PRs.
- It waits for GitHub to finish computing mergeability.
- If the PR is non-mergeable or dirty, it slaps a `cascade-conflict` label and posts a “cascade paused” comment.
- If the PR is mergeable (or at least not explicitly conflict/dirty), it just logs that and exits.

To check your understanding: in your own words, what is the advantage of using `pull_request_target` here instead of `pull_request` for this conflict-check workflow?

**

1. `cascade-conflict-check.yml`
2. <https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions>
3. <https://eastondev.com/blog/en/posts/dev/20260405-github-actions-yaml-basics/>